DWEC · 2º DAW · UNIDAD 3 · PROYECTOS

Proyectos integradores

Un gestor de incidencias del aula: formulario, una clase, cookies, localStorage y temporizadores

Desarrollo Web en Entorno Cliente · apartados 3.2 a 3.5

PROYECTO INTEGRADOR · 3.2 – 3.5

Un gestor de incidencias del aula, en tres pasos

1 ≈ 45 min

Lee y prueba el ejemplo

Lee tú las diapositivas siguientes con app.js abierto y pruébalo: guarda incidencias, recarga y mira dónde se guardan.

2 ≈ 1 h

Tres retos

Rellenas los huecos // TODO del código. Cada reto tiene su «Deberías ver» para comprobarlo.

3 10 min

Demuéstralo sin IA

Un pequeño cambio en tu código y dos preguntas sobre cómo funciona.

Qué usarás de cada apartado

2.6
Eventos
addEventListener

1.8
Pintar
innerHTML · textContent

3.2
Tiempo
Date · setTimeout · setInterval

3.3
Cookies
recordar el nombre

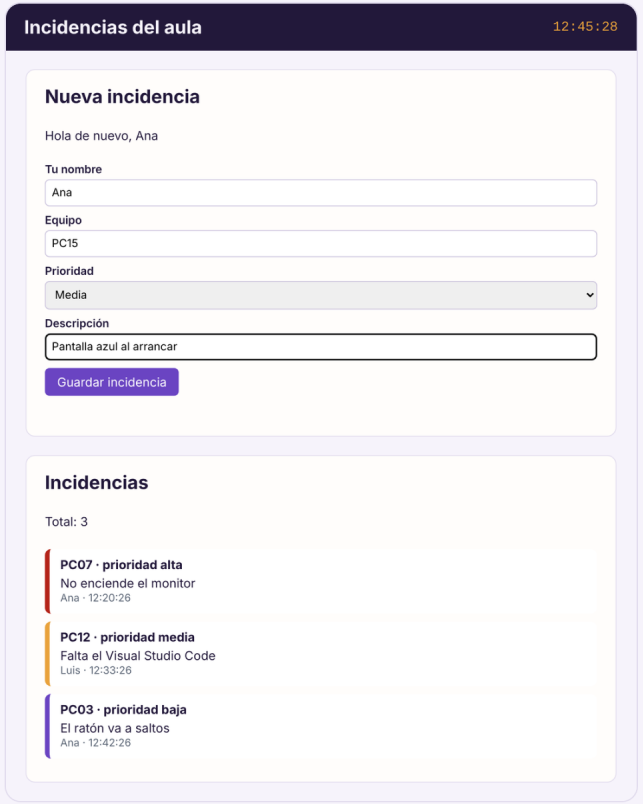
3.4
localStorage
guardar el array con JSON

3.5
Objetos
una clase con un método

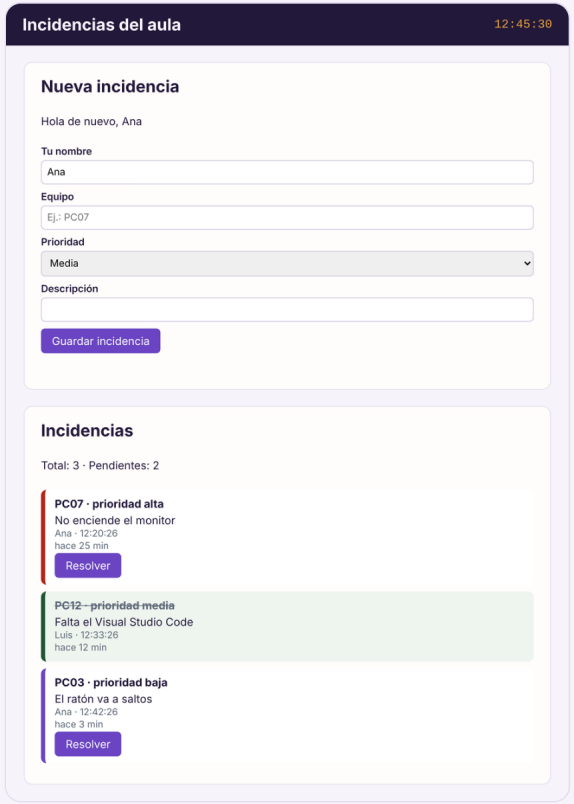
PROYECTO · ASÍ SE VE

El programa en el navegador

Al empezar: rellenas el formulario y pulsas Guardar



Al terminar los retos: Resolver, pendientes y «hace X min»



PROYECTO · CÓMO FUNCIONA

Del formulario a la lista, en cuatro pasos

1 · Formulario
Pulsas el botón: se leen los campos con .value

→

2 · Objeto
new Incidencia(...) crea el objeto con sus datos

→

3 · Array
push lo añade al array incidencias

→

4 · Guardar y pintar
guardar() lo pasa a localStorage; pintar() rehace la lista

Al abrir la página se hace al revés: **cargar()** lee localStorage, vuelve a crear cada Incidencia y **pintar()** la muestra. Por eso, si recargas, no se pierde nada.

3 ficheros: index.html, estilos.css y app.js, donde está todo el código.

Ábrelo con Live Server: con file:// las cookies no funcionan.

EJEMPLO · 1/4

La página: campos con id, un botón y la lista

```
<label>Equipo <input id="equipo"></label>
<label>Prioridad
  <select id="prioridad">
    <option value="alta">Alta</option> …
  </select>
</label>
<label>Descripción <input id="descripcion"></label>
<button type="button" id="enviar">Guardar</button>
<p id="aviso"></p> <!— mensajes —>
<p id="resumen"></p> <!— Total: 4 —>
<div id="lista"></div> <!— aquí se pinta —>
<script src="app.js"></script>
```

Cada campo, un id

Desde JavaScript se busca con getElementById y su valor se lee con .value

El script al final

Así, cuando se ejecuta app.js, los elementos ya existen.

EJEMPLO · 2/4

La clase Incidencia: datos y un método html()

```
class Incidencia {
  constructor(autor, equipo, prioridad, descripcion) {
    this.id = Date.now(); // número único (ms desde 1970)
    this.fecha = Date.now(); // cuándo se creó
    this.autor = autor; // … equipo, prioridad y descripcion igual
    this.resuelta = false;
  }
  html() { // devuelve el HTML de su tarjeta
    let clases = "incidencia " + this.prioridad;
    if (this.resuelta) clases += " resuelta";
    let html = "<article class='" + clases + "'>";
    html += "<h3>" + this.equipo + " · prioridad " + this.prioridad + "</h3>";
    html += "<p>" + this.descripcion + "</p>";
    return html + "</article>";
  }
}
```

EJEMPLO · 3/4

Guardar y cargar el array con localStorage

```
let incidencias = [];

function guardar() {
  localStorage.setItem("incidencias", JSON.stringify(incidencias));
}

function cargar() {
  const texto = localStorage.getItem("incidencias");
  if (texto === null) return; // la primera vez no hay nada
  const datos = JSON.parse(texto); // vuelve como objetos SIN métodos
  for (let i = 0; i < datos.length; i++) {
    const inc = new Incidencia(datos[i].autor, datos[i].equipo,
                               datos[i].prioridad, datos[i].descripcion);
    inc.id = datos[i].id; // … y fecha y resuelta igual
    incidencias.push(inc);
  }
}
```

EJEMPLO · 4/4

El botón: leer, comprobar, crear, guardar y pintar

```
document.getElementById("enviar").addEventListener("click", () ⇒ {
  const equipo = document.getElementById("equipo").value.trim();
  const descripcion = document.getElementById("descripcion").value.trim();
  // … autor y prioridad igual
  if (equipo ═══ "" || descripcion ═══ "") {
    avisar("Rellena todos los campos.");
    return;
  }
  document.cookie = "autor=" + encodeURIComponent(autor) + "; max-age=2592000";
  incidencias.push(new Incidencia(autor, equipo, prioridad, descripcion));
  guardar();
  pintar();
  avisar("Incidencia guardada ✓"); // se borra a los 2 s con setTimeout
});
```

Al abrir la página: leerCookie("autor") rellena tu nombre · setInterval actualiza el reloj · cargar() y pintar().

</> PRUÉBALO · EL EJEMPLO

Úsalo antes de tocarlo

1. Guarda dos incidencias
   aparecen en la lista y el aviso se borra solo
2. Pulsa el botón con un campo vacío
   «Rellena todos los campos.»
3. F12 › Application › Local Storage
   ahí está tu array, en texto JSON
4. Recarga la página (F5)
   las incidencias siguen y te saluda por tu nombre
5. En la consola: **localStorage.clear()** y recarga
   la lista se queda vacía

Reto rápido: haz que el aviso tarde 4 segundos en borrarse.

Pregúntate: ¿qué línea hace que la lista siga ahí al recargar?

EJERCICIO · ≈ 1 H · EN CLASE

Tres retos sobre el mismo código

En **app.js** busca los comentarios **// TODO RETO**: cada uno dice dónde escribir y qué hacer.

Reto 1 — ≈ 25 min

Resolver

Añade el botón «Resolver» en html() y programa resolver(id): busca con un for la incidencia con ese id, ponle resuelta = true, guarda y pinta.

Deberías ver

La tarjeta sale tachada, en verde y sin botón. Al recargar, sigue resuelta.

Reto 2 — ≈ 15 min

Pendientes

En pintar(), cuenta con un for las incidencias que no están resueltas y añádelo al resumen.

Deberías ver

«Total: 4 · Pendientes: 3». Al resolver una, pasa a 2.

Reto 3 — ≈ 20 min

¿Hace cuánto?

Programa el método minutos() (Date.now() – this.fecha, en minutos) y muéstralo en cada tarjeta.

Deberías ver

«hace 0 min» al crearla; espera un par de minutos, recarga y verás «hace 2 min».

CIERRE · 10 MIN · SIN IA

Demuestra que es tu código

Un cambio en vivo

El profesor te pedirá un pequeño cambio en tu propio código y lo harás en ese momento.

Dos preguntas

Te preguntará cómo funciona alguna parte del programa. Respuesta corta, con tus palabras.

¿Cómo prepararlo? Entiende cada línea de app.js: qué hace, por qué está ahí y qué pasaría si la quitas. Si la ha escrito otro (o una IA) y no la entiendes, pregunta en clase.

40 %	**30 %**	**20 %**	**10 %**
Retos 1 a 3 funcionan	Cambio en vivo	Dos preguntas	Código limpio